



fitsync.

GYM & WELLNESS MANAGEMENT
PLATFORM

Group Member:

Name: Aarij Farhan, Seat No: B22110006001
Name: Ahshan Khan, Seat No: B22110006012
Name: Rayyan Sohail, Seat No: B22110006108
Name: Moazzam Ali, Seat No: B22110006077

SOFTWARE ARCHITECTURE & SYSTEM DOCUMENTATION

FitSync — Fitness Gym Management & Member Wellness Platform

A role-based, multi-tenant web application unifying gym administration, trainer coaching workflows, member attendance, workout tracking, and AI-assisted diet planning behind a single Free / Pro subscription model.

[Live Demo: https://fit-sync-gym-management-fitsync.vercel.app/](https://fit-sync-gym-management-fitsync.vercel.app/)

TypeScript

JavaScript

React

Node.js

Express.js

MongoDB

JWT

bcrypt

OpenAI API

Project Overview, Problem & Objectives

1.1 PROJECT OVERVIEW

FitSync is a multi-tenant SaaS web application built to run the day-to-day operations of a fitness gym from a single system. It is organized around four role-based experiences — Super Admin, Gym Owner, Trainer, and Member — each with a purpose-built dashboard, so that administrative, coaching, and member-facing needs are served by the same underlying platform without exposing one role's data to another. The application is implemented as a TypeScript/JavaScript full-stack system with a React client, a Node.js/Express API, and a MongoDB data layer, and is deployed live on Vercel.

1.2 PROBLEM STATEMENT

Independent gyms and small fitness studios typically coordinate members, trainers, and attendance through spreadsheets, physical registers, and disconnected chat threads. This produces four recurring issues: attendance and check-ins that are not reliably recorded; no single view of a member's goals, workouts, and progress; no structured, personalized way to deliver nutrition guidance; and no live visibility for an owner into revenue, active trainers, or membership growth. Because each gym's data must also remain private from every other gym on the platform, a shared system needs strict tenant-level isolation rather than a single flat database.

1.3 OBJECTIVES

Operational Objectives

- › Give every role a dashboard scoped to what it needs to see and act on
- › Digitize attendance capture with a timestamped, searchable history
- › Let trainers log workouts and follow member progress in one place
- › Surface owner-level metrics: members, trainers, revenue, attendance, growth

Platform Objectives

- › Enforce strict multi-tenant isolation between gyms
- › Secure every role behind authentication and access control
- › Gate advanced functionality (AI diet plans, growth analytics) behind a Pro subscription
- › Keep the platform's Free tier genuinely useful for a small, single gym

1.4 SCOPE

The current system covers member and trainer management, attendance recording, workout logging and scheduling, an AI-assisted diet plan generator, a Free/Pro billing tier managed through Stripe, and a platform-level Super Admin view across all gym tenants. Scope is intentionally limited to what is implemented and verifiable from the shipped product: it does not include capabilities such as native mobile apps, wearable-device integration, or in-app messaging, which are not part of the current build.

In scope vs. out of scope. Everything described in this document reflects functionality observed in the deployed application and the project's own feature documentation. Capabilities are not extrapolated beyond what the system demonstrably does.

System Overview, User Roles & Functional Modules

2.1 SYSTEM OVERVIEW

FitSync is delivered as a single web application whose interface and available actions change based on the signed-in user's role. Every gym that registers becomes its own tenant: its members, trainers, workouts, attendance records, and diet plans are scoped to that gym and are not visible to any other gym. A platform-level Super Admin sits above all tenants and can view aggregate metrics and manage tenant status, but day-to-day gym operation is handled entirely by the Gym Owner, Trainer, and Member roles within their own tenant.

2.2 USER ROLES

Super Admin

PLATFORM

- View every registered gym on the platform
- Check each gym's Free/Pro subscription status
- Track total platform-wide revenue
- See total members and active gyms across all tenants
- Suspend or reactivate a gym account

Gym Owner

TENANT ADMIN

- Manage their gym's members and trainers
- Add and search members; track progress and attendance
- Manage workout sessions and monitor revenue/growth
- Manage the Free/Pro plan and upgrade to Pro
- View dashboard: members, trainers, revenue, attendance, growth chart, activity log

Trainer

STAFF

- View their assigned members only
- Check member goals and fitness progress
- Log completed workouts; add workout type & duration
- Record member attendance

Member

END USER

- View workout schedule and history
- Track attendance history and progress streaks
- Check gym subscription status
- Generate a personalized diet plan (breakfast/lunch/dinner/snacks)

2.3 FUNCTIONAL MODULES

MODULE	WHAT IT DOES	PRIMARY ROLES
Member Directory	Add, search, and filter members; view goal, assigned coach, status, and progress %.	Owner, Trainer
Trainer Management	Add trainers; view specialty, active/away status, and assigned member count.	Owner
Attendance	Quick check-in by member or member ID; searchable Present/Absent history with timestamps.	Owner, Trainer, Member
Workouts	Log and schedule sessions by type and duration; Scheduled/Completed status per session.	Trainer, Member
Diet Plans	Collects age, weight, height, gender, goal, activity level and restrictions to generate a 7-day meal plan.	Member
Billing / Subscription	Free vs. Pro plan comparison and Stripe-based upgrade flow (\$79/month Pro).	Owner
Owner Dashboard	Total members, active trainers, monthly revenue, attendance rate, growth chart, recent activity feed.	Owner
Platform Admin	Cross-tenant visibility: all gyms, plan status, total revenue, suspend/activate accounts.	Super Admin

Non-functional characteristics. Role-based access control on every module; per-tenant data isolation enforced for all reads/writes; stateless authentication so the API can be called from any signed-in client; a demo-mode fallback for AI generation so the diet module remains usable without a live OpenAI key.

System Architecture

FitSync follows a logical client / API / data separation. The diagram below shows the layers visible from the implementation: a React presentation layer, a Node.js/Express API layer enforcing authentication and tenant scoping, a MongoDB data layer, and the external services the API integrates with.

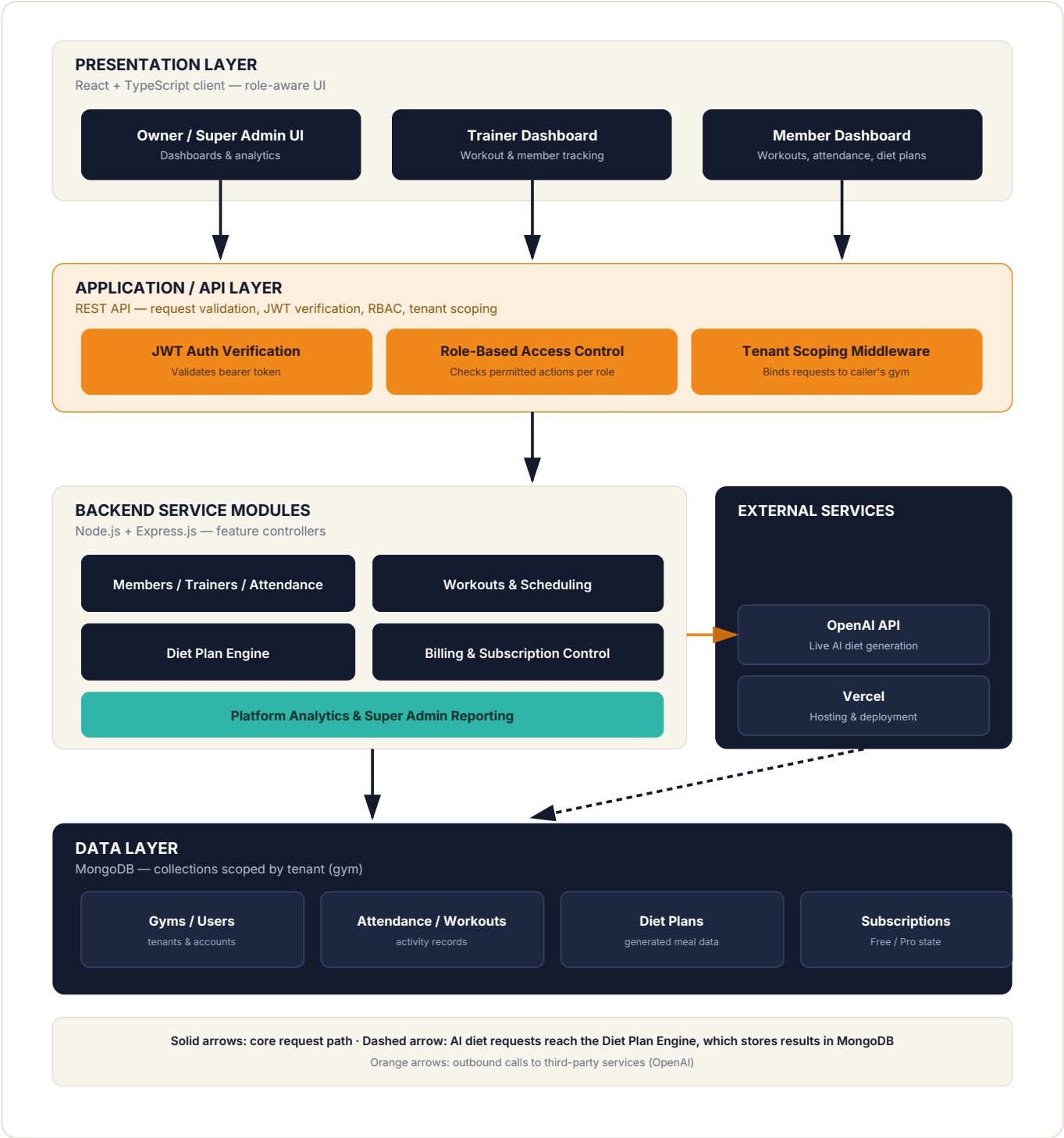


Figure 1 — FitSync high-level system architecture (Presentation → API/Application → Backend Services / External Services → Data)

Use Case Diagram

Use cases are grouped by responsibility rather than broken into individual UI actions, so that each bubble represents a coherent unit of functionality actually available to that role.

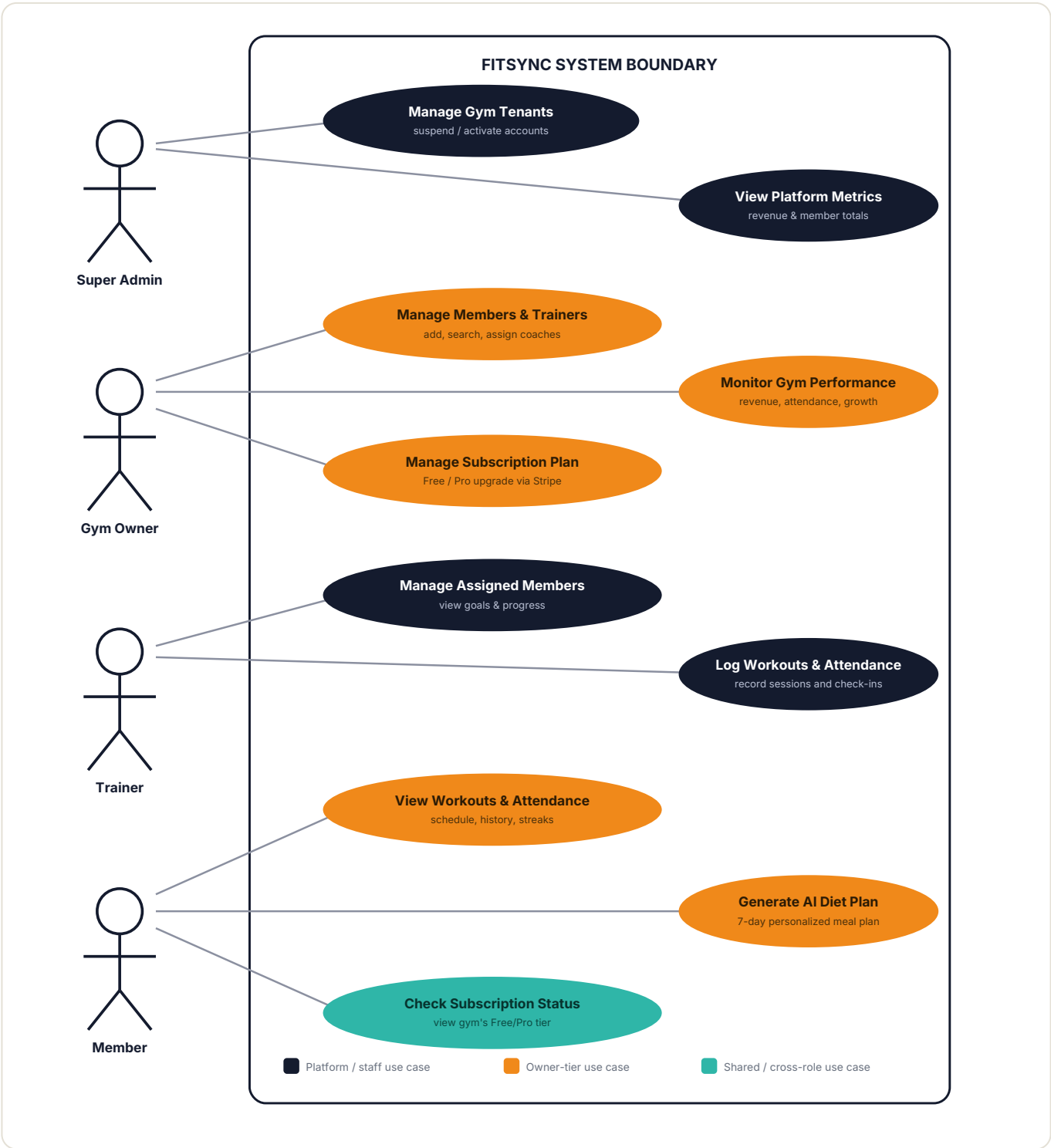


Figure 2 — FitSync use case diagram grouped by role responsibility (Super Admin, Gym Owner, Trainer, Member)

Entity Relationship Diagram (Conceptual)

The database schema is not exposed by the application, so the model below is presented as a **conceptual data model** — it captures the entities and relationships implied by the visible feature set (multi-tenant gyms, users with roles, attendance, workouts, diet plans, and subscriptions), not a verified physical collection schema.

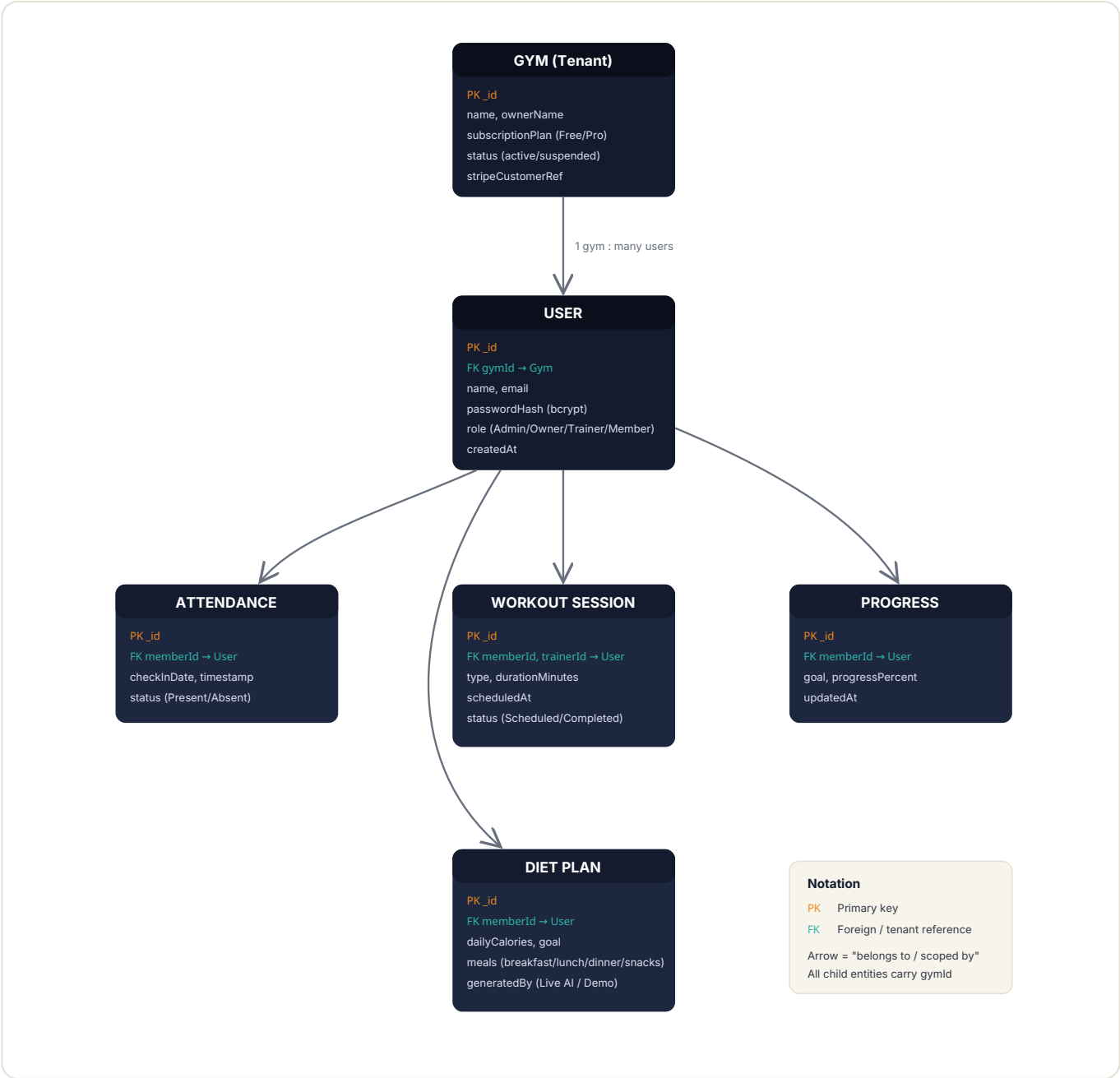


Figure 3 — Conceptual entity relationship model. Every entity beneath User is implicitly scoped to the owning Gym for tenant isolation.

System Flow Diagram

Two representative flows: how an authenticated API request is processed end-to-end, and how the AI diet generator decides between a live OpenAI call and its built-in demo fallback.

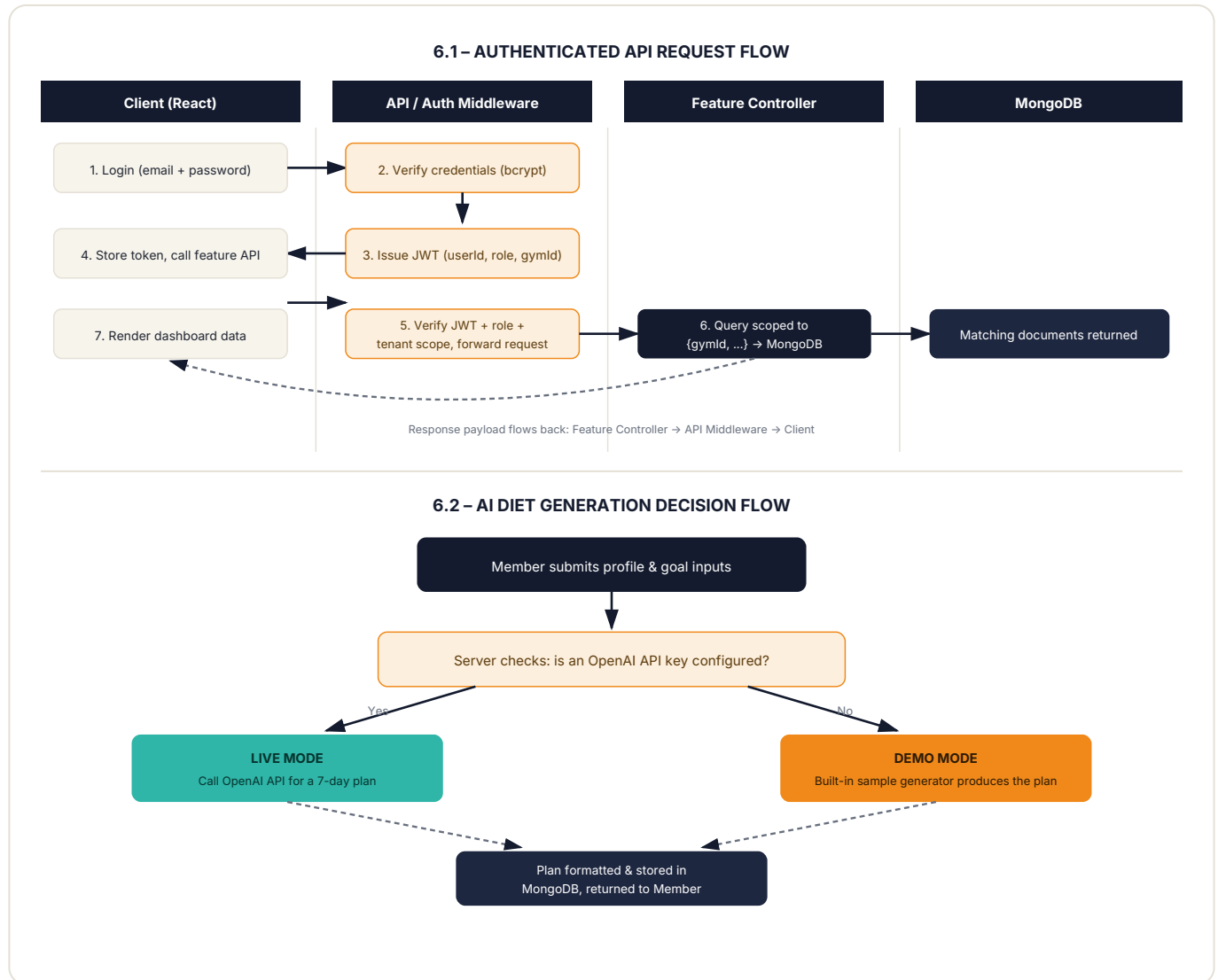


Figure 4 — End-to-end authenticated request flow (6.1) and the Live/Demo branching logic behind AI diet generation (6.2)

Technology, Security, Subscriptions & Conclusion

7.1 TECHNOLOGY STACK

LAYER	TECHNOLOGY	ROLE IN THE SYSTEM
Frontend	React TypeScript JavaScript	Role-aware single-page client rendering the Owner, Trainer, Member, and Super Admin dashboards.
Backend	Node.js Express.js	REST API layer: routing, request handling, and feature controllers per module.
Database	MongoDB	Document store for gyms, users, attendance, workouts, diet plans, and subscription state.
Auth & Security	JWT bcrypt	Stateless session tokens and salted password hashing behind Role-Based Access Control.
Payments	Stripe	Handles the Free → Pro subscription upgrade flow.
AI Services	OpenAI API	Generates live, personalized diet plans when an API key is configured.
Deployment	Vercel	Hosts the deployed, production build of the application.

The codebase is organized as a TypeScript-first monorepo (pnpm workspaces, shared tsconfig base), consistent with a project that is roughly 88% TypeScript and 7% JavaScript by source volume.

7.2 AUTHENTICATION & SECURITY

Identity & Access

- › JWT-based authentication issued at login
- › Passwords stored as bcrypt password hashes, never in plain text
- › Role-Based Access Control across Super Admin, Gym Owner, Trainer, and Member

Multi-Tenancy

- › Every gym's data is isolated within its own tenant scope
- › A Gym Owner can only manage their own gym's members and trainers
- › The Super Admin alone holds platform-wide, cross-tenant visibility

7.3 SUBSCRIPTION ARCHITECTURE

PLAN	INCLUDED
Free — \$0/mo	Member directory, attendance tracking, workout scheduling, a basic analytics dashboard, and a limited member capacity. The AI diet generator is not included.
Pro — \$79/mo	Everything in Free, plus unlimited members, AI-powered diet plans, advanced growth analytics, priority features, and the Stripe-based upgrade flow that unlocks them.

7.4 AI DIET GENERATOR

Members provide age, weight, height, gender, fitness goal, activity level, and dietary restrictions. The system returns a 7-day meal plan broken down into breakfast, lunch, dinner, and snacks, with total calorie tracking aligned to the stated goal. **Live mode** calls the OpenAI API when a key is configured; otherwise the platform falls back to a **built-in demo generator** so the feature stays usable without a live key.

7.5 Deployment

FitSync is deployed and running in production on Vercel, serving the React client and the Node.js/Express API described throughout this document.

7.6 Conclusion

FitSync delivers a working, role-based, multi-tenant gym management platform: four distinct dashboards, attendance and workout tracking, a Free/Pro subscription model, and an AI-assisted diet planner with a reliable demo fallback — implemented and shipped, not merely specified.